

fptemplate

fptemplate.pp has been moved to fcl-base.

fptemplate.pp

implements template support

Default behaviour: In the basic default version the TFPTemplate object can handle simple template tags ex. {templatetagname} and requires the replacement strings in a Values array before the parsing starts. An OnGetParam:TGetParamEvent event can be triggered if it is set, when a value is not found in the Values list.

The template tag start and end delimiters can be set with the StartDelimiter and EndDelimiter properties (defaults are '{' and '}' for now).

The parsing happens recursively so a replace text string can contain further tags in it.

Recent improvements: With the recent improvements the template tag handling got more close to the traditional Delphi way of handling templates. The rest of this file is about these additions. By setting the AllowTagParams property to True this new parsing method will be activated and it is possible to pass parameters to the processing program from the template tags.

Other than the two original StartDelimiter and EndDelimiter properties to specify the boundaries of a template tag, there are 3 more delimiters to define these parameters. ParamStartDelimiter (default is '[') ParamEndDelimiter (default is ']') ParamValueSeparator (default is '=')

Some examples for tags with these above, StartDelimiter:='{+' and EndDelimiter:='}+' (the default '{' and '}' is not good when processing HTML templates with JavaScript in them):

```
{+ATagHere+} //Tag name: ATagHere
```

```
{+AnotherTagHere [-paramname1=paramvalue1-]+}
```

```
{+HereIsATagToo //Tag name=HereIsATagToo; with 3 paramname-paramvalue pairs [-  
param1=param1value-] //some text here to ignore //this text is ignored too [-  
param2=param2value which is multi line something text ending here -] [-  
param3=param3value-] +}
```

If we want something close to the Delphi tag delimiters, we can set the StartDelimiter := '<#'; EndDelimiter := '>'; ParamStartDelimiter := ' '; ParamEndDelimiter := ' '; ParamValueSeparator := '=';

This allows the use of Delphi-like tags like these:

```
<#input type="text" name="foo1" value="" caption="bar" checked="false"> <#input  
type="RadioButton" name="foo2" value="" caption="bar" checked="false" > <#fieldvalue
```

```
fieldname="FIRSTNAME">
```

Of course, the above setting requires at least one space before the parameter names. Cannot just use tabs for example to separate them. Also, Delphi (and its emulation here) cannot handle any HTML code within the tag parameters because some might contain characters indicating tag-param-end or tag-end (see below to overcome this).

When the tags are processed, for each tag a

```
TReplaceTagEvent = Procedure(Sender : TObject; Const TagString : String;  
TagParams:TStringList; Out ReplaceText : String) Of Object;
```

will be called with the parameters passed in TagParams:TStringList so it has to be assigned to such a procedure.

Example:

```
procedure TFPWebModule1.func1callRequest(Sender: TObject; ARequest: TRequest;  
AResponse: TResponse; var Handled: Boolean); var s:String; begin //Template:TFPTemplate  
is a property of the web module Template.FileName := 'pathtotemplate\mytemplate.html';  
Template.AllowTagParams := true; Template.StartDelimiter := '{+'; Template.EndDelimiter :=  
'+'; Template.OnReplaceTag := @func1callReplaceTag; s := Template.GetContent;
```

```
//lets use some Delphi style tags too and re-run the parser Template.StartDelimiter := '<#';  
Template.EndDelimiter := '>'; Template.ParamStartDelimiter := ' ';  
Template.ParamEndDelimiter := ''; Template.ParamValueSeparator := '=';  
Template.FileName := ''; Template.Template := s;
```

```
AResponse.Content := Template.GetContent;
```

```
Handled := true; end;
```

```
procedure TFPWebModule1.func1callReplaceTag(Sender: TObject; const TagString: String;  
TagParams: TStringList; Out ReplaceText: String); begin if AnsiCompareText(TagString,  
'ATagHere') = 0 then begin ReplaceText := 'text to replace this tag, using the TagParams if  
needed'; end else begin . .snip . //Not found value for tag -> TagString end; end;
```

With these improvements it is easily possible to separate the web page design and the web server side programming. For example to generate a table record list the web designer can use the following Tag in a template:

```
. .snip .
```

```
{+REPORTRESULT [-HEADER= -] [- ONEROW = -] . .snip, and so on more parameters if  
needed . [- NOTFOUND= -] [-FOOTER=
```

~Column1	~Column2
~Column1Value	~Column2value

There are no entries found.

```
-] +}
```

```
. .snip .
```

I know, I know its ugly html programming and who uses tables and font html tags nowadays, etc. ... but you get the idea. The OnReplaceTag event handler just need to replace the whole REPORTRESULT template tag with the ~Column1, ~Column2 for the HEADER parameter, and the ~Column1Value, ~Column2Value in the ONEROW parameter while looping through a sql query result set. Or if there is nothing to list, just use the NOTFOUND parameter as a replace text for the whole REPORTRESULT template tag.

Sample code for the above template snippet (see below for simpler examples in the Step By Step list):

```
procedure TFPWebModule1.func2callRequest(Sender: TObject; ARequest: TRequest;
AResponse: TResponse; var Handled: Boolean); var s:String; begin //Template:TFPTemplate
is a property of the web module Template.FileName := 'pathtotemplate\mytemplate.html';
{template file with the above template tag -> REPORTRESULT} Template.AllowTagParams :=
true; Template.StartDelimiter := '{+'; Template.EndDelimiter := '+'; Template.OnReplaceTag
:= @func2callReplaceTag;
```

```
AResponse.Content := Template.GetContent;
```

```
Handled := true; end;
```

```
procedure TFPWebModule1.func2callReplaceTag(Sender: TObject; const TagString: String;
TagParams: TStringList; Out ReplaceText: String); var header, footer, onerow,
notfound:String; NoRecordsToShow, EndOfRecords:Boolean; begin//HTML template tag
handling for an html template file //Replace the REPORTRESULT html tag using it's tag
parameters if AnsiCompareText(TagString, 'REPORTRESULT') = 0 then begin
//NoRecordsToShow could be something like SQL1.IsEmpty , etc. if NoRecordsToShow then
begin //if there's nothing to list, just replace the whole tag with the //Not Found message
that the template contains ReplaceText := TagParams.Values['NOTFOUND']; Exit; end;
```

```
header := TagParams.Values['HEADER'];
//insert header parameters
//1st column title
header := StringReplace(header, '~Column1', '1st column', []);
//2nd column title
header := StringReplace(header, '~Column2', '2nd column', []);

ReplaceText := header;//done with the header (could have been looping
//through table field names also)
//insert the rows
onerow := TagParams.Values['ONEROW'];//template for 1 row
//loop through the rows, it could be someting like "while not SQL1.EOF
do"
while not EndOfRecords do
begin
    ReplaceText := ReplaceText + StringReplace(StringReplace(onerow
        , '~Column1Value', '$14.56', [])
```

```
        , '~Column2Value', '$12.00', []));

    //get the next record, it could be:
    //SQL1.Next
end;

//insert the footer
footer := TagParams.Values['FOOTER'];
//replace footer parameters if needed
//...

ReplaceText := ReplaceText + footer;
```

end else begin

//Not found value for tag -> TagString ReplaceText := 'Template tag {' + TagString + '}' is not implemented yet.'; end; end;

full example code at </lazarus/components/fpweb/demo/fptemplate/listrecords/>

Step by Step:

Creating CGI or Apache applications with WebModule in Lazarus, using HTML templates (FPTemplate)

I. Hello World first II. Using templates III. More complicated HTML template design notes IV. Passing tag parameters

I. Hello World first:

1. File -> New -> CGI application or Apache module
2. Delete the httpd20 and httpd13 directories (we are making Apache 2.2 modules) from the fpc directory (ex: C:\pp\units\i386-win32\httpd20\ and C:\pp\units\i386-win32\httpd13) Need to recompile FPC and then Lazarus if FPC was earlier compiled with these older httpd13 or httpd20 files. To avoid this recompilation you can also just copy all the files from the /packages/fcl-web/src/ directory into your project directory so they will be recompiled as needed.
3. Click inside the webmodule if not already selected
4. In the Object Inspector double click on the "Actions"
5. Click on +Add to create a new action for your web module

6. Change Default to True if you wish this one to be the default action
7. Change the action name to "func1call" (this will be the calling identifier of this action from the web browser. Something like http://localhost/mod_apache1/func1call?param1=...)
8. Inside the Events tab, double click on the "OnRequest" to create the procedure called "func1callRequest" that handles this action
9. Enter the following into the procedure body:

```
procedure TFPWebModule1.func1callRequest(Sender: TObject; ARequest: TRequest;
AResponse: TResponse; var Handled: Boolean); begin
AResponse.Content := 'Hello World! ';
```

```
Handled := true; end;
```

10. Save all, compile, configure the apache server to load the module: in your apache httpd.conf you can put

```
LoadModule mod_apache1 "/mod_apache1.dll" #or mod_apache1.so <Location
/myapache> SetHandler mod_apache1 Order allow,deny Allow from all
```

11. Call your module action from your web browser ex:
<http://localhost/myapache/func1call?param1=paramvalue1>
12. See "Hello World!" in your browser
13. Repeat from step 4 for other web actions

full example code at /lazarus/components/fpweb/demo/fptemplate/helloworld/

II. Using templates:

1. Lets make a simple html template and save it as mytemplate1.html :

This is a replaced template tag: {TagName1}

2. Save it and put it somewhere your apache module can access it (ex: below the apache module .dll or .so in a directory called "templates/")
3. Declare a procedure for your web module to handle the template tags

```
private { private declarations } procedure func1callReplaceTag(Sender: TObject; const
TagString:String; TagParams: TStringList; Out ReplaceText: String);
```

4. Create the body of the procedure

```
procedure TFPWebModule1.func1callReplaceTag(Sender: TObject; const TagString: String;
TagParams: TStringList; Out ReplaceText: String); begin if AnsiCompareText(TagString,
'TagName1') = 0 then begin ReplaceText := 'Here I am from the web module!'; end else
begin
```

```
//Not found value for tag -> TagString ReplaceText := 'Template tag {' + TagString + '}' is not implemented yet.'; end; end;
```

5. In step 9 above in the first example change the procedure body to:

```
procedure TFPWebModule1.func1callRequest(Sender: TObject; ARequest: TRequest;
AResponse: TResponse; var Handled: Boolean); begin //Template:TFPTemplate is a property
of the web module Template.FileName := 'pathtotemplate/mytemplate1.html';
Template.AllowTagParams := true; Template.OnReplaceTag := @func1callReplaceTag;
AResponse.Content := Template.GetContent;

Handled := true; end;
```

6. Compile, etc. and call it. Should show

This is a replaced template tag: Here I am from the web module!

full example code at /lazarus/components/fpweb/demo/fptemplate/ simpletemplate/

III. More complicated HTML template design notes:

1. Template tag delimiters.

Template.StartDelimiter := '{+'; Template.EndDelimiter := '+}'; should be used if there are { or } characters in the HTML template (ex: Javascript exist in the template)

2. For "same as Delphi" template tag handling, use Template.StartDelimiter := '<#';
Template.EndDelimiter := '>'; Template.ParamStartDelimiter := ' ';
Template.ParamEndDelimiter := ' '; Template.ParamValueSeparator := '=';

ex: <#TagName1 param1="value1" param2="value2">

```
=====
==
```

IV. Passing tag parameters:

You can pass parameters to your CGI/Apache web module from the templates.

Example HTML template tag: {+HereIsATag [-param1=param1value-] //some text here to ignore -] [-param3=param3value-] +}

ex: {+DATETIME [-FORMAT=MM/DD hh:mm:ss-]+}

Code: procedure TFPWebModule1.func1callRequest(Sender: TObject; ARequest: TRequest; AResponse: TResponse; var Handled: Boolean); var s:String; begin //Template:TFPTemplate is a property of the web module Template.FileName := 'pathtotemplate\templatename.html'; Template.AllowTagParams := true;

```
Template.StartDelimiter := '{+'; Template.EndDelimiter := '+'; Template.OnReplaceTag :=  
@func1callReplaceTag;
```

```
AResponse.Content := Template.GetContent;
```

```
Handled := true; end;
```

```
procedure TFPWebModule1.func1callReplaceTag(Sender: TObject; const TagString: String;  
TagParams: TStringList; Out ReplaceText: String); begin if AnsiCompareText(TagString,  
'DATETIME') = 0 then begin ReplaceText := FormatDateTime(TagParams.Values['FORMAT'],  
Now); end else begin
```

```
//Not found value for tag -> TagString ReplaceText := 'Template tag {' + TagString + '}' is not  
implemented yet.'; end; end;
```

For example, this way if the web designer changes the look of a page, - in this case the format of the date/time on the page - no changes are needed in the apache module code, therefore no recompiling or apache restart is needed. The best way is to make the project such, that the web/html design is separated from the back end apache module as much as possible.

full example code at </lazarus/components/fpweb/demo/fptemplate/tagparam/>

